

Manipulating and Visualizing Data

Tables, joins, and graphics in R and Python



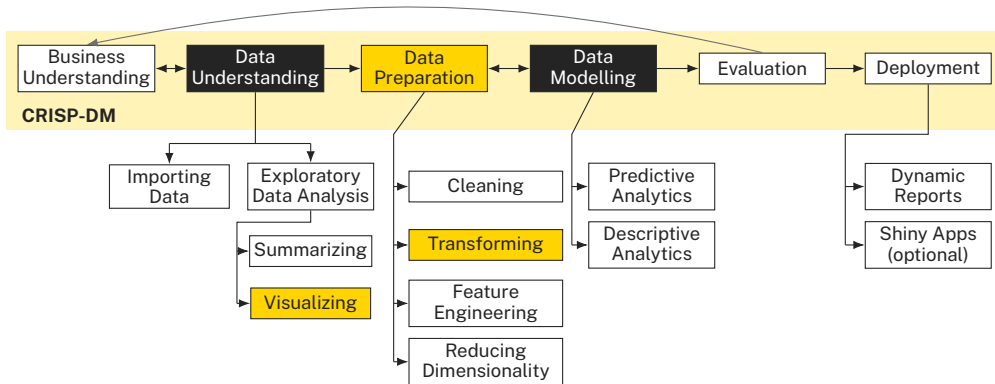
Prof. Gabriel Spadon, PhD

Faculty of Computer Science
spadon@dal.ca

From Exploration to a Report

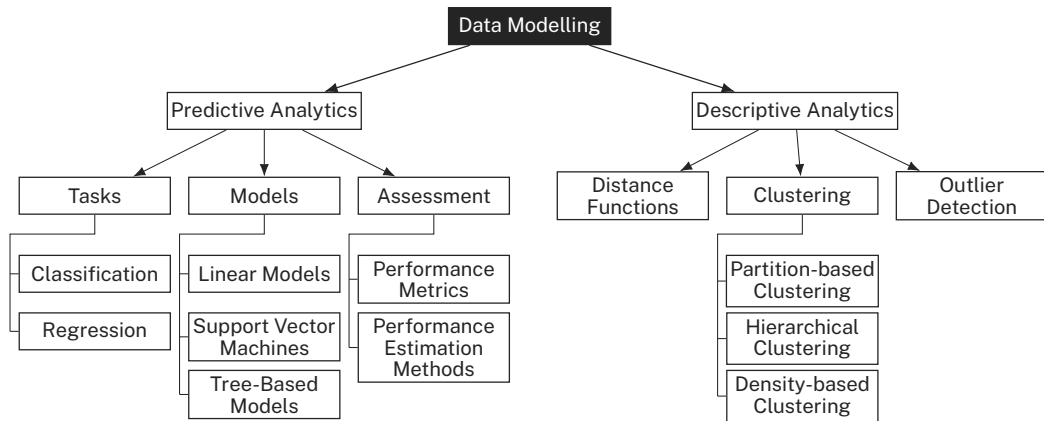
- Week 3 paired charts with numerical summaries and checked the meaning of their axes.
- Today we make the table operations behind those displays explicit.
- Class B combines code, results, and interpretation in a reproducible report.
- Every example is shown in R and Python side by side, on the same records.

Manipulation and Visual Exploration



We move between preparing a useful table and examining what it shows.

Where the Prepared Results Go



Reporting is the next step this week. Models and their evaluation are taught later.

From Records to an Explained Figure

- Distinguish a tibble, a data frame, and a pandas column.
- Select, reorder, derive, and summarize values, including within groups.
- Combine tables while checking keys, row counts, and missing values.
- Draw and save a labelled figure using base R or Matplotlib, then connect it to the grammar of graphics.

Start with the Meaning of a Row

A table operation changes the available evidence.

Our game example has one record per player–game pair. A second table supplies a genre and an achievement total for each listed game.

A Tibble Extends a Data Frame

- A tibble inherits from `data.frame`, but has its own printing and subsetting methods.
- Selecting one column with `tb[, "x"]` keeps a table. Explicit `tb[["x"]]` extracts a vector.
- In pandas, `tb[["x"]]` is a `DataFrame` and `tb["x"]` is a `Series`.
- Check a function's documented input behaviour when moving between table classes.

Load the Libraries for Your Language

R

```
library(dplyr)  
library(ggplot2)
```

Python

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.datasets import load_iris
```

Run the examples in order in a working folder where output files can be saved.

Six Verbs and Their Effects

<code>filter</code>	Keep rows that meet a condition
<code>select</code>	Keep or reorder columns
<code>arrange</code>	Reorder rows
<code>mutate</code>	Add, replace, or remove columns
<code>summarise</code>	Return a summary row per group
<code>group_by</code>	Define groups without removing rows

Grouping records does not by itself collapse them. A later summary determines the output rows.

Eight Records and a Game Lookup

- Each record is one player and one game.
- Dan has one unknown hours value.
- Tunic appears in the play records but has no lookup entry.
- Stardew appears only in the lookup table.

Keep the original tables. Derived tables will make each decision visible.

Create the Play Records

R

Player-game records.

```
plays <- tibble(  
  player = c("ana", "ana", "ben", "ben",  
             "cara", "cara", "dan", "dan"),  
  game = c("Portal", "Celeste", "Portal",  
           "Hades", "Celeste", "Hades",  
           "Portal", "Tunic"),  
  hours = c(12.5, 30, 8, 45.5, 22, 17.5, NA, 6),  
  achieved = c(9L, 14L, 4L, 25L, 11L, 8L,  
              2L, 3L))
```

Each row is one player-game record. Hours measure the time played, and achieved counts the achievements earned.

Python

Player-game records.

```
plays = pd.DataFrame({  
    "player": ["ana", "ana", "ben", "ben",  
              "cara", "cara", "dan", "dan"],  
    "game": ["Portal", "Celeste", "Portal",  
            "Hades", "Celeste", "Hades",  
            "Portal", "Tunic"],  
    "hours": [12.5, 30, 8, 45.5, 22, 17.5,  
             np.nan, 6],  
    "achieved": [9, 14, 4, 25, 11, 8, 2, 3]})
```

Create the Lookup Table

R

```
# Game lookup.
games <- tibble(
  game = c("Portal", "Celeste", "Hades",
           "Stardew"),
  genre = c("puzzle", "platformer",
            "roguelike", "farming"),
  total = c(15L, 30L, 49L, 40L))
print(plays)
print(games)
```

Python

```
# Game lookup.
games = pd.DataFrame({
    "game": ["Portal", "Celeste", "Hades",
            "Stardew"],
    "genre": ["puzzle", "platformer",
             "roguelike", "farming"],
    "total": [15, 30, 49, 40]})
print(plays)
print(games)
```

Continue from the play-record setup. Each game key is unique in this lookup.

Select Records

R

```
print(filter(plays, hours > 20))  
print(filter(plays, hours > 20,  
            player != "ana"))  
print(filter(plays,  
            game %in% c("Hades", "Tunic")))
```

The first condition retains Ana–Celeste, Ben–Hades, and Cara–Celeste. Comma-separated conditions in `filter` are combined with AND; pandas keeps the original index labels.

Python

```
print(plays.loc[plays["hours"] > 20])  
print(plays.query(  
    "hours > 20 and player != 'ana'"))  
print(plays.loc[plays["game"].isin(  
    ["Hades", "Tunic"])]])
```

Make the Selection Rule Explicit

- R comparisons with NA produce an unknown result. `filter` keeps only true conditions.
- For the floating-point NaN in this Python example, `NaN > 20` is false but `NaN != 20` is true.
- Pandas nullable `pd.NA` follows different comparison rules. Do not generalize from one missing-value representation.

Decide whether unknown values belong in the analysis, then include that decision in the mask.

Check the Missing-Value Policy

R

```
print(nrow(filter(plays, hours > 20))) # 3
print(nrow(filter(plays, hours != 20))) # 7
base_plays <- as.data.frame(plays)
print(nrow(
  base_plays[base_plays$hours > 20, ])) # 4
known <- filter(plays, !is.na(hours),
               hours != 20)
print(nrow(known)) # 7, explicit NA policy
```

Base bracket selection carries an all-NA row for the unknown condition; `filter` keeps only TRUE. The explicit known-hours selection gives seven records in both languages.

Python

```
print(len(plays.loc[plays["hours"] > 20])) # 3
print(len(plays.loc[plays["hours"] != 20])) # 8
known = plays.loc[
    plays["hours"].notna()
    & (plays["hours"] != 20)]
print(len(known)) # 7, explicit NA policy
```

Select and Rename Variables

R

```
print(select(plays, player, game, hours))  
print(select(plays, -achieved))  
print(select(plays, player:hours))  
print(select(plays, where(is.numeric)))  
print(rename(plays, hrs = hours))
```

A label range includes both endpoints. An ordinary Python positional slice excludes its stop position.

Python

```
print(plays[["player", "game", "hours"]])  
print(plays.drop(columns="achieved"))  
print(plays.loc[:, "player":"hours"])  
print(plays.select_dtypes("number"))  
print(plays.rename(columns={"hours": "hrs"}))
```

Order Records

R

```
print(arrange(plays, desc(hours)))  
print(arrange(plays, player, desc(achieved)))
```

Python

```
print(plays.sort_values(  
    "hours", ascending=False,  
    kind="stable", na_position="last"))  
print(plays.sort_values(  
    ["player", "achieved"],  
    ascending=[True, False]))
```

Missing hours appear last. Add ordering columns when ties matter; in pandas, `kind="stable"` keeps the input order of ties.

Derive a Rate

R

```
rated <- mutate(plays,  
  denominator = if_else(hours > 0, hours,  
                        NA_real_),  
  rate = achieved / denominator,  
  band = if_else(rate > 0.5, "hi", "lo"))  
print(select(rated, player, game, rate, band))
```

Rate is achievements per known positive hour. Missing or nonpositive hours produce a missing rate and band.

Python

```
rated = plays.assign(  
  denominator=lambda d: d.hours.where(  
    d.hours > 0),  
  rate=lambda d: d.achieved / d.denominator,  
  band=lambda d: pd.Series(  
    np.where(d.rate > 0.5, "hi", "lo"),  
    index=d.index, dtype="string",  
  ).where(d.rate.notna()))  
print(rated[["player", "game", "rate", "band"]])
```

Report the Summary Denominator

R

```
print(mean(plays$hours)) # NA
overview <- summarise(plays,
  records = n(),
  observed_hours = sum(!is.na(hours)),
  mean_hours = mean(hours, na.rm = TRUE),
  achievements = sum(achieved))
print(overview)
# 8 records, 7 observed, 20.214286 h, 76
```

There are four players, eight play records, and seven observed hours values. The mean describes those seven values.

Python

```
print(plays["hours"].mean(skipna=False)) # nan
overview = pd.DataFrame([{
  "records": len(plays),
  "observed_hours": plays["hours"].count(),
  "mean_hours":
    plays["hours"].mean(skipna=True),
  "achievements": plays["achieved"].sum()})]
print(overview)
# 8 records, 7 observed, 20.214286 h, 76
```

Read a Pipeline from Top to Bottom

In R, `x |> f()` passes `x` as the first argument of `f`.

A pandas method chain passes the resulting table to the next method.
Parentheses allow the chain to span lines.

Choose groups, calculate one summary for each group, then drop the grouping before passing that result on.

Summarize Each Player

R

```
by_player <- plays |>
  group_by(player) |>
  summarise(records = n(),
    observed_hours = sum(!is.na(hours)),
    known_hours = sum(hours, na.rm = TRUE),
    achievements = sum(achieved),
    .groups = "drop")
print(by_player)
```

Dan has six known hours from his Tunic record. His Portal record has unknown hours, so this is a partial observed sum.

Python

```
by_player = (plays
  .groupby("player", as_index=False)
  .agg(records=("game", "size"),
    observed_hours=("hours", "count"),
    known_hours=("hours", "sum"),
    achievements=("achieved", "sum")))
print(by_player)
```

Keep the Original Record Count

R

```
shares <- plays |>
  group_by(player) |>
  mutate(share_known_hours =
    hours / sum(hours, na.rm = TRUE)) |>
  ungroup()
print(select(shares, player, game,
  share_known_hours))
```

Group-wise mutation or transform returns eight records. A summary aggregation instead returns four player rows.

Python

```
shares = plays.assign(share_known_hours=(
    plays.hours
    / plays.groupby("player").hours
    .transform("sum")))
print(shares[["player", "game",
  "share_known_hours"]])
```

Control Group Order

R

```
dan_first <- arrange(plays, desc(player))  
print(summarise(dan_first,  
  achievements = sum(achieved), .by = player))  
# First appearance: dan, cara, ben, ana.
```

Python

```
dan_first = plays.sort_values(  
    "player", ascending=False)  
print(dan_first.groupby("player", sort=False,  
    as_index=False)  
    .agg(achievements=("achieved", "sum")))  
# First appearance: dan, cara, ben, ana.
```

The reordered input begins with Dan. First-appearance groups are dan, cara, ben, ana; default grouped summaries sort keys.

Retain an Unobserved Category

R

```
levels_player <- c("ana", "ben", "cara",  
                  "dan", "eve")  
p <- mutate(plays,  
  player = factor(player,  
                  levels = levels_player))  
empty_groups <- p |>  
  group_by(player, .drop = FALSE) |>  
  summarise(records = n(), .groups = "drop")  
print(empty_groups) # eve has 0 recorded rows
```

Eve is a possible category with no recorded rows. Absence from these records does not establish zero real-world activity.

Python

```
levels_player = ["ana", "ben", "cara", "dan",  
                 "eve"]  
p = plays.assign(player=pd.Categorical(  
  plays.player, categories=levels_player))  
empty_groups = (p  
  .groupby("player", observed=False,  
           as_index=False)  
  .agg(records=("game", "size")))  
print(empty_groups) # eve has 0 recorded rows
```

Compose the Six Operations

R

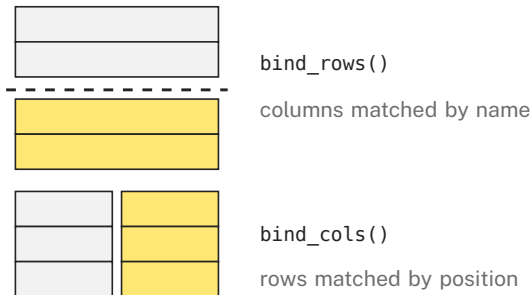
```
player_rates <- plays |>
  filter(!is.na(hours), hours > 0) |>
  mutate(rate = achieved / hours) |>
  group_by(player) |>
  summarise(records = n(),
            mean_rate = mean(rate),
            .groups = "drop") |>
  arrange(desc(mean_rate)) |>
  select(player, records, mean_rate)
print(player_rates)
```

The mean gives every eligible player–game record equal weight. It is not total achievements divided by total hours.

Python

```
player_rates = (plays
                .loc[plays.hours.notna()
                    & (plays.hours > 0)]
                .assign(rate=lambda d: d.achieved / d.hours)
                .groupby("player", as_index=False)
                .agg(records=("game", "size"),
                    mean_rate=("rate", "mean"))
                .sort_values("mean_rate", ascending=False)
                [[ "player", "records", "mean_rate"]])
print(player_rates)
```

Append Rows or Add Aligned Columns



Appending rows aligns column names. Adding columns requires an established correspondence between rows.

Append Compatible Records

R

```
extra <- tibble(game = "Tunic",  
                genre = "puzzle", total = 42L)  
extended <- bind_rows(games, extra)  
print(extended) # games itself still has 4 rows  
a <- tibble(x = 1)  
b <- tibble(y = 2)  
print(bind_rows(a, b)) # missing cols become NA
```

Appending does not update the original lookup variable. Check column names, units, and types before binding.

Python

```
extra = pd.DataFrame({"game": ["Tunic"],  
                      "genre": ["puzzle"], "total": [42]})  
extended = pd.concat([games, extra],  
                     ignore_index=True)  
print(extended) # games itself still has 4 rows  
a = pd.DataFrame({"x": [1]})  
b = pd.DataFrame({"y": [2]})  
print(pd.concat([a, b], ignore_index=True))
```

Check Row Correspondence

R

```
x <- tibble(id = c("a", "b"), value = c(2, 3))
y <- tibble(id = c("a", "b"),
            label = c("low", "high"))
stopifnot(identical(x$id, y$id))
print(bind_cols(x, select(y, label)))
```

The identifiers match in the same order. Check this correspondence before attaching columns by position, and do not reset unrelated pandas indexes to force a match.

Python

```
x = pd.DataFrame({"id": ["a", "b"],
                  "value": [2, 3]})
y = pd.DataFrame({"id": ["a", "b"],
                  "label": ["low", "high"]})
assert x["id"].equals(y["id"])
print(pd.concat([x, y[["label"]]], axis=1))
```

Track Matched and Unmatched Records

7 matched
play records
Portal, Celeste,
and Hades

1 left-only record
Dan's Tunic play

1 right-only game
Stardew
lookup row

`inner_join` 7 matched records

`left_join` $7 + 1 = 8$ records, including Tunic

`right_join` $7 + 1 = 8$ rows, including Stardew

`full_join` $7 + 1 + 1 = 9$ rows, including both

These counts depend on one lookup row per game. Duplicate lookup keys would create extra matched combinations.

Validate the Lookup Relationship

R

```
stopifnot(!anyNA(plays$game),  
          !anyNA(games$game))  
stopifnot(!anyDuplicated(games$game))  
joined <- left_join(plays, games, by = "game",  
  relationship = "many-to-one",  
  na_matches = "never")  
stopifnot(nrow(joined) == nrow(plays))  
print(joined)
```

Complete keys and a unique lookup key are checked before the merge. The left join preserves all eight play records.

Python

```
assert plays["game"].notna().all()  
assert (games["game"].notna().all()  
        and games["game"].is_unique)  
joined = plays.merge(games, on="game",  
                    how="left",  
                    validate="many_to_one")  
assert len(joined) == len(plays)  
print(joined)
```

Compare the Four Join Counts

R

```
print(nrow(joined))           # 8
print(nrow(inner_join(plays, games,
                      by = "game")) # 7
print(nrow(right_join(plays, games,
                      by = "game")) # 8
print(nrow(full_join(plays, games,
                     by = "game")) # 9
```

Python

```
print(len(joined))           # 8
print(len(plays.merge(games, on="game",
                      how="inner"))) # 7
print(len(plays.merge(games, on="game",
                      how="right"))) # 8
print(len(plays.merge(games, on="game",
                      how="outer"))) # 9
```

Inner has seven rows; left and right each have eight; full or outer has nine.

Make Cardinality an Explicit Expectation

Check	dplyr	pandas
Lookup cardinality	<code>relationship = "many-to-one"</code>	<code>validate="many_to_one"</code>
Missing keys	May match by default; <code>na_matches="never"</code> prevents this	May match by default; check or separate missing keys before merging
Output order	Left rows first, then unmatched right rows	Outer merge sorts keys; sort explicitly if order matters
Name clashes	Default suffixes <code>.x</code> , <code>.y</code>	Default suffixes <code>_x</code> , <code>_y</code>

Validate the lookup relationship and inspect unmatched records.

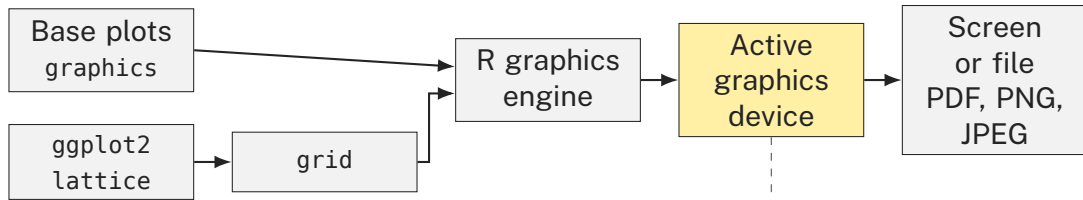
Draw from a Checked Table

The figure should answer a question already defined by the table and its eligibility rules.

—

We revisit the Iris measurements from Week 3 to compare plotting interfaces, then return to the game summary.

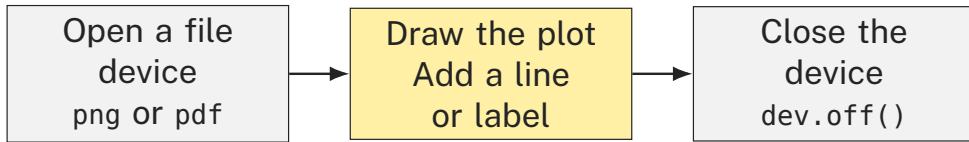
From Drawing Commands to an Output



grDevices provides functions to open, select, and close devices.

Base graphics and grid use the graphics engine and an active output device.

Open, Draw, and Close



Drawing commands go to the active device.
Closing the file device completes that output.

Interactive sessions may draw on screen; a script may choose another default.
Select the intended file device explicitly.

Load the Iris Measurements

R

```
d <- as_tibble(iris)
print(nrow(d))           # 150 flowers
print(mean(d$Sepal.Width)) # 3.057333 cm
```

Python

```
raw = load_iris(as_frame=True)
d = raw.data.copy()
d.columns = ["Sepal.Length", "Sepal.Width",
             "Petal.Length", "Petal.Width"]
d["Species"] = pd.Categorical.from_codes(
    raw.target, raw.target_names)
print(len(d), d["Sepal.Width"].mean())
# 150, 3.057333 cm
```

The same 150 flower measurements and centimetre units used in Week 3 are available locally in both languages.

Save the Scatter Plot

R

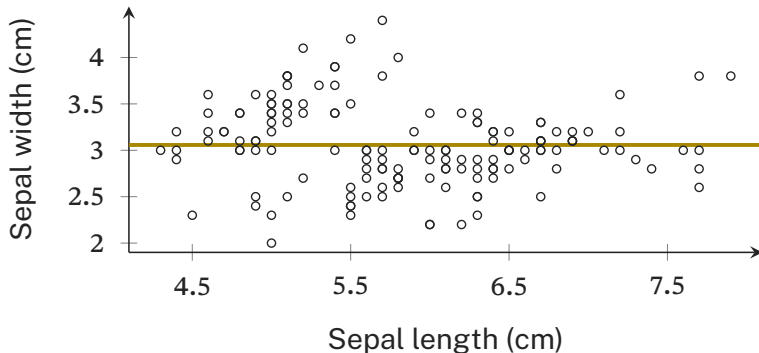
```
png("class07-iris-base.png", width = 1200,  
    height = 800, res = 150)  
plot(d$Sepal.Length, d$Sepal.Width, pch = 1,  
     xlab = "Sepal length (cm)",  
     ylab = "Sepal width (cm)")  
abline(h = mean(d$Sepal.Width),  
       col = "#A88800", lwd = 2)  
dev.off()
```

Python

```
fig, ax = plt.subplots(figsize=(8, 16 / 3))  
ax.scatter(d["Sepal.Length"], d["Sepal.Width"],  
          facecolors="none",  
          edgecolors="black")  
ax.axhline(d["Sepal.Width"].mean(),  
           color="#A88800")  
ax.set(xlabel="Sepal length (cm)",  
       ylabel="Sepal width (cm)")  
fig.savefig("class07-iris-base.png", dpi=150)  
plt.close(fig)
```

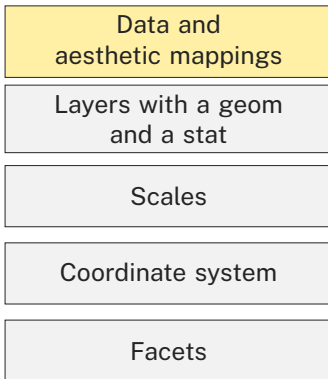
The gold line marks the overall mean sepal width, 3.057333 cm. The PNG is 1200 by 800 pixels.

Show the Overall Mean as a Reference



Measured to 0.1 cm, the 150 flowers share 117 positions, so some overlap. The gold line is the mean width, not a fitted relationship.

Separate the Choices That Make a Plot



A point layer displays the original values.

A histogram layer counts values in bins.

Defaults supply many settings, but data and mappings alone draw no marks.

The grammar describes how data, marks, calculations, and display settings work together. See Wilkinson and the ggplot2 book in the notes.

Connect Each Element to the Iris Plot

Element	Meaning	Iris example
Data	Values to display	Flower measurements
Aesthetic mappings	Variables linked to visible properties	Length to x , width to y
Geometric objects	Marks that display values	One point per flower
Statistical transformations	Calculations used by a layer	Identity for points; counts for histogram bins

A layer combines marks with a calculation.

Choose How the Values Are Displayed

Element	Meaning	Iris example
Scales	Convert values into positions, colours, or other properties	Numeric axes and species colours
Coordinate system	Geometry of the plotting panel	Cartesian x and y
Faceting	Separate panels for subsets	One panel per species

A scale need not be one-to-one. Several values can share a binned colour category.

Draw One Panel per Species

R

```
colours <- c("#242424", "#A88800", "#6E6E6E")
p_iris <- ggplot(d,
  aes(Sepal.Length, Sepal.Width,
      colour = Species)) +
  geom_point() +
  geom_hline(yintercept = mean(d$Sepal.Width),
            colour = "#A88800") +
  facet_wrap(~ Species) +
  scale_colour_manual(values = colours) +
  labs(x = "Sepal length (cm)",
       y = "Sepal width (cm)") +
  theme_minimal()
ggsave("class07-iris-facets.pdf", p_iris,
      width = 9, height = 3.5)
```

Python

```
colours = ["#242424", "#A88800", "#6E6E6E"]
sns.set_theme(style="whitegrid")
g = sns.relplot(
    data=d, x="Sepal.Length", y="Sepal.Width",
    hue="Species", col="Species",
    kind="scatter", palette=colours,
    height=3.5, aspect=6 / 7)
g.refline(y=d["Sepal.Width"].mean(),
          color="#A88800")
g.set_axis_labels("Sepal length (cm)",
                  "Sepal width (cm)")
g.savefig("class07-iris-facets.pdf")
plt.close(g.figure)
```

Each species has its own labelled panel. Every panel uses the overall width mean as the same gold reference.

Check Eligibility Before Summarizing

- Return to the play records and their checked left join.
- This question concerns records with known hours greater than five and a known positive achievement total.
- Retain the two excluded rows for inspection. Do not let a grouping default decide which evidence disappears.
- Summarize the six eligible records, then draw that summary.

Retain an Exclusion Record

R

```
checked <- mutate(joined,  
  eligible = !is.na(hours) & hours > 5 &  
    !is.na(total) & total > 0 & !is.na(genre))  
excluded <- filter(checked, !eligible)  
eligible <- filter(checked, eligible)  
print(select(excluded, player, game, hours,  
  genre, total))  
print(c(input = nrow(plays),  
  eligible = nrow(eligible),  
  excluded = nrow(excluded))) # 8, 6, 2
```

Dan-Portal is excluded because hours are unknown. Dan-Tunic is excluded because its game lookup and denominator are missing.

Python

```
checked = joined.assign(eligible=  
  joined.hours.notna() & (joined.hours > 5) &  
  joined.total.notna() & (joined.total > 0) &  
  joined.genre.notna())  
excluded = checked.loc[~checked.eligible].copy()  
eligible = checked.loc[checked.eligible].copy()  
print(excluded[["player", "game", "hours",  
  "genre", "total"]])  
print(len(plays), len(eligible),  
  len(excluded)) # 8, 6, 2
```

Compute the Genre Summary

R

```
genre_summary <- eligible |>
  mutate(pct = 100 * achieved / total) |>
  group_by(genre) |>
  summarise(records = n(),
            known_hours = sum(hours),
            pct = mean(pct),
            .groups = "drop") |>
  arrange(desc(pct))
print(genre_summary)
```

Each eligible play record receives equal weight. All three genres have two records.

Python

```
genre_summary = (eligible
                 .assign(pct=lambda d:
                        100 * d.achieved / d.total)
                 .groupby("genre", as_index=False)
                 .agg(records=("player", "size"),
                     known_hours=("hours", "sum"),
                     pct=("pct", "mean"))
                 .sort_values("pct", ascending=False))
print(genre_summary)
```

Save the Genre Figure

R

```
p_genre <- ggplot(genre_summary,  
  aes(reorder(genre, -pct), pct)) +  
  geom_col(fill = "#FFD400",  
    colour = "#242424") +  
  geom_text(aes(label = round(pct, 1)),  
    vjust = -0.4) +  
  scale_y_continuous(limits = c(0, 50)) +  
  labs(x = "Genre",  
    y = "Mean achievement completion (%)") +  
  theme_minimal()  
ggsave("class07-genre-summary.pdf", p_genre,  
  width = 7, height = 4)
```

Python

```
fig, ax = plt.subplots(figsize=(7, 4))  
bars = ax.bar(genre_summary.genre,  
  genre_summary.pct,  
  color="#FFD400",  
  edgecolor="#242424")  
ax.bar_label(bars, fmt="%.1f")  
ax.set(xlabel="Genre",  
  ylabel="Mean achievement completion (%)",  
  ylim=(0, 50))  
fig.tight_layout()  
fig.savefig("class07-genre-summary.pdf")  
plt.close(fig)
```

Both languages draw the same ordered bar chart with its labelled values and save it as a PDF.

Save the Summary and the Exclusions

R

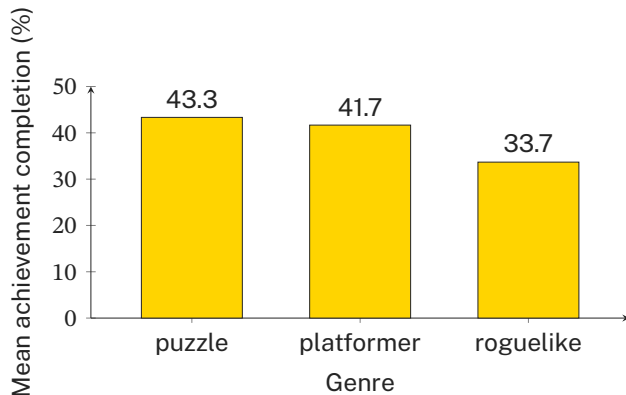
```
write.csv(genre_summary,  
          "class07-genre-summary.csv",  
          row.names = FALSE)  
write.csv(excluded, "class07-excluded.csv",  
          row.names = FALSE)
```

The saved CSV, chart, and exclusion record provide inputs for a report. Keep the source tables and the code as well.

Python

```
genre_summary.to_csv(  
    "class07-genre-summary.csv", index=False)  
excluded.to_csv("class07-excluded.csv",  
               index=False)
```

Read the Mean Completion Percentage



Six eligible records, with two records in each genre.

Make Every Transformation Explainable

- Say what each row represents and how many values a summary uses.
- Keep original data and retain a record of exclusions.
- Validate lookup keys and distinguish unmatched rows from duplicated matches.
- Save a labelled figure with its numerical summary and the code that produced both.

Bring the Analysis into a Report

- Class B introduces R Markdown and Python notebooks for combining text, code, results, and figures.
- As ungraded practice, rerun the example in your chosen language and explain the two exclusions.
- Lab 04 uses the datasets and evidence-brief requirements in its own current instructions.
- Keep assessed work private until the deadline and instructor release. Preserve the submitted commit and its history.

Continue in Your Chosen Language

- [dplyr, Introduction](#) and [pandas, Introductory tutorials](#). Revisit selection, derived columns, and grouped summaries in your chosen language.
- [dplyr, Mutating joins](#) or [pandas, Combining tables](#). Check keys and expected row relationships.
- [Wickham, Navarro, and Pedersen, Building a plot layer by layer](#). Connect each grammar element to a visible choice.
- [plotnine, Introduction](#) and [seaborn, Objects interface](#). Optional Python alternatives for composing graphical layers.